ADAPTIVE HIERARCHICAL MEMORY NETWORKS FOR INTELLIGENT DOCUMENT PROCESSING SYSTEMS

Dasanayake D R L
IT22350800

Dissertation submitted in partial fulfilment of the requirements for the
Bachelor of Science (Honours) in Information Technology
Specializing in Data Science

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

April 2025

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning, and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute my dissertation, in whole or in part, in print, electronic or other medium. I retain the right to use this content in whole or in part in future works (such as articles or books).

Signature: _____ Date: _____

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature of Supervisor: _____ Date: _____

ABSTRACT

Automated document processing systems frequently struggle to adapt to new document types and changing vendor templates. To address this, this dissertation presents the Adaptive Hierarchical Memory Network (AHMN). The AHMN is a three-tier memory architecture comprising Short-Term, Mid-Term, and Long-Term storage that allows a multi-agent system to independently learn, retain, and forget processing policies, mirroring human memory consolidation.

Two novel algorithms drive this architecture. First, Retrieval-Augmented Bandit Initialization (RABI) solves the "cold-start" problem. When encountering an unseen document type, RABI transfers prior knowledge from semantically similar historical documents rather than starting from scratch. Second, Drift-Aware Adaptive Forgetting (DAAF) manages non-stationary environments. DAAF distinguishes between normal processing variations and actual structural changes in documents, allowing the system to calculate an optimal rate to forget outdated rules. Together, these mechanisms enhance a Thompson Sampling framework to balance the exploration of new processing strategies with the exploitation of known successful ones.

The AHMN was evaluated against four baseline algorithms across stationary, cold-start, changing, and combined environments. Operating within a broader 13-agent pipeline featuring safety guardrails and human-in-the-loop feedback, the system was evaluated across four experimental scenarios that defined the precise conditions under which each mechanism provides measurable benefit. The results demonstrate that the complete AHMN system significantly reduces errors in cold-start and drifting environments, while maintaining strong, comparable performance in stable conditions.

Keywords: Adaptive Hierarchical Memory Networks, Thompson Sampling, Cold-Start Problem, Concept Drift, Multi-Agent Document Processing

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor and co-supervisor for the guidance, patience, and constructive feedback provided throughout this research project. Their insights into machine learning systems and adaptive decision-making frameworks significantly shaped the direction and depth of this work.

I am grateful to the Department of Information Technology at the Sri Lanka Institute of Information Technology for providing an environment that encourages independent research and technical exploration. The facilities and resources made available during this study were invaluable to the development and evaluation of the proposed system.

I would also like to thank my fellow students and peers who engaged in discussions about algorithms, memory architectures, and document processing systems throughout the year. These conversations helped me refine my ideas and identify gaps in my thinking that I might otherwise have overlooked.

Finally, I extend my deepest appreciation to my family for their unwavering support and encouragement during the demanding process of completing this dissertation. Their patience and understanding made this work possible.

TABLE OF CONTENTS

Declaration i
Abstract ii
Acknowledgements iii
List of Tables v
List of Figures vi
List of Abbreviations vii
1. Introduction 1
1.1 Background Literature 1
1.2 Research Gap 6
1.3 Research Problem 8
1.4 Research Objectives 10
2. Methodology 15
2.1 System Architecture 15
2.2 Adaptive Hierarchical Memory Network (AHMN) 19
2.2.1 Short-Term Memory (STM) - Redis 19
2.2.2 Mid-Term Memory (MTM) - PostgreSQL mtm_outcomes 21
2.2.3 Long-Term Memory (LTM) - PostgreSQL with pgvector 23
2.3 Algorithmic Implementation 29
2.3.1 Thompson Sampling for Policy Selection 29
2.3.2 RABI Cold-Start Transfer 34
2.3.3 Drift-Aware Adaptive Forgetting (DAAF) 39
2.3.4 Human Feedback (via /advice/feedback) 46
2.3.5 Review Feedback (via /advice/review) 47
2.4 Supporting Subsystems 48
2.5 Commercialisation Aspects 51
3. Results and Discussion 55
3.1 Performance Optimisations 55
3.2 Evaluation Metrics 58
3.3 Discussion 62
3.4 Summary of My Contribution 66
4. Conclusion 69
Future Recommendations 71
References 73
Appendix A: Configuration Reference 78
A.1 AHMNConfig vs Actual Runtime Values 78
A.2 Production Environment Variables 79
A.3 RABI Production Parameters 79

A.4 DriftEstimator Parameters **80**
A.5 Scheduler Parameters **80**

LIST OF TABLES

Table 2.1 Mid-Term Memory (MTM) Schema - mtm_outcomes table 23
Table 2.2 Long-Term Memory (LTM) Schema - ltm_validated_policies table 25
Table 3.1 Evaluation Scenarios and Conditions 47

LIST OF FIGURES

Figure 2.1 AHMN Three-Tier Memory Architecture 21
Figure 2.2 RABI Softmax-Weighted Pseudocount Transfer 31
Figure 2.3 DAAF Dual Sliding-Window Drift Estimation 35

LIST OF ABBREVIATIONS

Abbreviation	Description
AHMN	Adaptive Hierarchical Memory Network
API	Application Programming Interface
BART	Bidirectional and Auto-Regressive Transformers
BERT	Bidirectional Encoder Representations from Transformers
CLS	Complementary Learning Systems
CORS	Cross-Origin Resource Sharing
DAAF	Drift-Aware Adaptive Forgetting
EHR	Electronic Health Records
EWC	Elastic Weight Consolidation
F1	F1 Score - harmonic mean of precision and recall
GRU	Gated Recurrent Unit
HIPAA	Health Insurance Portability and Accountability Act
HITL	Human-in-the-Loop
HTTP	Hypertext Transfer Protocol
IDPO	Intelligent Document Processing Orchestration
JSON	JavaScript Object Notation
JSONB	JSON Binary (PostgreSQL binary-encoded JSON)
LIME	Local Interpretable Model-agnostic Explanations
LLM	Large Language Model
LSTM	Long Short-Term Memory
LTM	Long-Term Memory
MTM	Mid-Term Memory
OCR	Optical Character Recognition
PII	Personally Identifiable Information
RABI	Retrieval-Augmented Bandit Initialisation
RNN	Recurrent Neural Network
SHA	Secure Hash Algorithm
STM	Short-Term Memory

TTL	Time-to-Live
UCB1	Upper Confidence Bound 1
UUID	Universally Unique Identifier
XAI	Explainable Artificial Intelligence

1. INTRODUCTION

1.1 Background Literature

The processing of unstructured and semi-structured documents has long presented a fundamental challenge for automated systems. Early approaches to sequential decision-making in such environments relied on recurrent neural networks (RNNs) and their gated variants - Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRU). These architectures were designed to capture temporal dependencies by maintaining hidden state vectors across time steps. However, RNNs suffer from vanishing and exploding gradients during backpropagation through time, which limits their ability to retain information over long sequences. While LSTMs partially mitigate this through gating mechanisms that regulate the flow of information, they still struggle to model very long-range dependencies and are inherently sequential in computation, making them difficult to scale [10], [3].

The introduction of the Transformer architecture by Vaswani et al. [23] marked a significant departure from recurrence-based models. Transformers rely entirely on self-attention mechanisms, enabling parallel computation and effective modelling of long-range dependencies within a fixed context window. This architecture underpins modern language models and has been adapted for document understanding tasks, as seen in LayoutLM [24], which integrates text, layout, and image embeddings for structured document comprehension. Sentence-level embeddings, such as those produced by Sentence-BERT [17], provide efficient semantic representations that have been widely adopted for retrieval and similarity tasks. The present work uses the all-MiniLM-L6-v2 model - a distilled Sentence-BERT variant producing 384-dimensional embeddings - for contextual policy retrieval within the AHMN's Long-Term Memory.

Despite these advances in representation learning, a deeper problem persists: catastrophic forgetting. Neural networks trained on one task tend to lose previously learned knowledge when fine-tuned on a new task [15], [7]. This phenomenon is well-documented in continual learning settings, where models must assimilate new information without overwriting old knowledge. Kirkpatrick et al. [11] proposed

Elastic Weight Consolidation (EWC), which selectively slows learning on parameters important to previous tasks. Complementary Learning Systems (CLS) theory from cognitive neuroscience [14] suggests that the brain addresses this through a dual-memory system: the hippocampus rapidly encodes new experiences while the neocortex slowly consolidates them into long-term knowledge. This biological insight directly informs the AHMN's three-tier memory hierarchy, where new processing outcomes are first captured in volatile Short-Term Memory (STM), aggregated in Mid-Term Memory (MTM), and selectively promoted to Long-Term Memory (LTM) only after demonstrating consistent reliability.

The decision of which processing strategy to apply to a given document type is naturally framed as a multi-armed bandit problem [18]. In this formulation, each available processing policy represents an "arm," and the system must learn which arm yields the highest reward - for example, extraction accuracy expressed as an F1 score - for a given context. Thompson Sampling [22], a Bayesian approach that maintains a posterior distribution over each arm's reward probability and selects arms by sampling from these posteriors, has been shown to achieve near-optimal regret bounds while naturally balancing exploration and exploitation [5], [19]. The Beta-Bernoulli variant, where each arm's success probability is modelled as a Beta distribution parameterised by counts of successes (alpha) and failures (beta), is particularly well-suited to the document processing domain, where outcomes can be characterised by continuous quality scores such as F1.

Upper Confidence Bound (UCB1) algorithms [2] offer an alternative frequentist approach with provable logarithmic regret bounds, selecting the arm that maximises an upper confidence bound computed from the empirical mean plus an exploration bonus proportional to the square root of log(t)/n. Epsilon-Greedy strategies [21] take a simpler approach, exploring randomly with probability epsilon and exploiting the current best arm otherwise, often with an exponential decay schedule on epsilon to reduce exploration over time. While these baselines provide useful comparison points, they lack the Bayesian flexibility that enables Thompson Sampling to incorporate prior knowledge - a property central to the RABI algorithm proposed in this work.

Hierarchical memory architectures for reinforcement learning and decision-making systems have been explored in various forms. Blundell et al. [4] proposed Model-Free Episodic Control, which stores experiences in differentiable memory and retrieves them for rapid value estimation. Guu et al. [9] introduced REALM, a retrieval-augmented language model that retrieves relevant documents from a knowledge store to condition generation - a paradigm that parallels the AHMN's retrieval of similar historical contexts from LTM to initialise priors for new document types. The key distinction in the present work is that the memory is not used for generation or feature augmentation, but for Bayesian prior initialisation in a bandit framework, creating a novel intersection of retrieval-augmented methods and online learning.

1.2 Research Gap

Two specific gaps in the existing literature motivate this research. These gaps are distinct in nature - one concerns the initial knowledge deficit when a new document type is encountered, while the other concerns the system's ability to remain accurate as document formats evolve over time.

The Cold-Start Problem in Contextual Bandits

Standard bandit algorithms initialise each new arm - or, in a contextual setting, each new context - with an uninformative prior, typically Beta(1,1), representing uniform uncertainty. This means the system must explore each new context-policy combination from scratch, incurring substantial regret during the exploration phase. In document processing, this translates directly to degraded accuracy when a new vendor, document format, or template is encountered for the first time. Existing approaches to cold-start in recommendation systems [20] and contextual bandits [12] often assume access to user-side or item-side features that can be used to generalise. However, they rarely exploit semantic similarity in a structured memory store to transfer full Bayesian posteriors from similar historical contexts.

The RABI algorithm proposed in this work fills this gap by retrieving the k-nearest neighbours from LTM using vector similarity over sentence embeddings, computing softmax-weighted prior pseudocounts, and initialising new arms with an informed Beta distribution rather than a flat one. Under Lipschitz continuity assumptions - where the true mean reward is a smooth function of the context embedding - this reduces cold-start regret from O(K) (where K is the number of arms) to O(L × d_avg), where L is the Lipschitz constant and d_avg is the average embedding distance to the neighbours used for transfer.

Concept Drift in Non-Stationary Environments

Real-world document processing environments are inherently non-stationary. Vendors update their invoice templates, regulatory changes alter form structures, and scanning hardware is upgraded or replaced - all of which shift the underlying reward distributions for different processing policies. Standard Thompson Sampling, and indeed most stationary bandit algorithms, accumulate evidence indefinitely, making

them slow to adapt when the true reward landscape changes. Existing work on non-stationary bandits includes discounted Thompson Sampling [16], sliding-window UCB [8], and change-point detection methods [13]. However, these approaches typically apply a fixed forgetting rate or a binary change-point trigger, without adapting the forgetting intensity to the magnitude and type of drift observed.

The DAAF mechanism proposed in this work addresses this by maintaining two sliding windows of recent rewards, computing the drift magnitude as the absolute difference between their means - which separates true distributional shift from within-window noise variance - and deriving an optimal forgetting rate from a bias-variance trade-off formulation. Specifically, the optimal rate gamma is approximately equal to the square root of (delta-squared divided by the natural logarithm of t), where delta is the estimated drift magnitude and t is the total number of observations. This provides a principled, continuously adaptive forgetting rate that responds proportionally to the severity of observed drift, with an additional amplification mechanism for abrupt changes.

1.3 Research Problem

Current document processing pipelines face several computational and architectural bottlenecks that limit their effectiveness in production settings. These bottlenecks are not merely technical inconveniences - they represent fundamental limitations in how automated systems learn from experience and adapt to change.

First, most systems apply a single, fixed processing strategy across all document types. A pipeline optimised for structured invoices with tabular data will underperform on scanned handwritten forms or mixed-content contracts. The policy selection problem - choosing the right combination of OCR engine, extraction method, and validation strategy for a given document - is typically hard-coded by engineers or determined through offline batch experiments that cannot adapt to changing conditions. This rigidity results in suboptimal processing accuracy and wasted computational resources when the chosen strategy is poorly suited to the input.

Second, the absence of a persistent memory mechanism means that systems cannot learn from their own processing history. When an identical vendor's invoice is processed for the hundredth time, the system applies the same selection logic it used for the first, gaining no benefit from the accumulated experience. There is no mechanism to record which strategy worked well for a given context, consolidate that evidence over time, or retrieve it when a similar context is encountered again. This lack of institutional memory forces repeated exploration and prevents the system from converging on reliable processing policies.

Third, in multi-agent architectures where multiple specialised agents - for OCR, entity extraction, validation, and structuring - collaborate on a document, the coordination overhead grows with system complexity. Without a centralised memory that captures end-to-end processing outcomes, each agent operates in isolation, unable to benefit from the system's collective experience. The 13-agent IDPO pipeline used in this research - encompassing intake, pre-processing, document analysis, categorisation, degarbling, chunking, retrieval, LLM extraction, structuring, validation, policy enforcement, escalation, and recovery agents - exemplifies this challenge.

Fourth, the system must handle the tension between retaining useful knowledge and discarding outdated information. A processing policy that worked well six months ago may be counterproductive today if the underlying document format has changed. Without an active forgetting mechanism, stale policies persist in the system's knowledge base, leading to overconfident but incorrect recommendations. Conversely, overly aggressive forgetting discards valuable experience prematurely, forcing unnecessary re-exploration.

These bottlenecks collectively define the research problem: how to design a memory-augmented decision system that (a) selects document processing policies adaptively based on context, (b) learns from processing outcomes and consolidates experience over time, (c) transfers knowledge to new, unseen document types to reduce exploration cost, and (d) forgets outdated knowledge at a rate proportional to the observed environmental change.

1.4 Research Objectives

The primary objectives of this research are as follows. Each objective is stated precisely enough to be tested through the evaluation framework described in Chapter 3.

Objective 1: Design and implement the three-tier AHMN architecture.

Design and implement a three-tier Adaptive Hierarchical Memory Network (AHMN) that organises document processing experience into Short-Term Memory (volatile, TTL-bounded storage in Redis for in-flight advice context), Mid-Term Memory (persistent outcome logs in PostgreSQL capturing policy performance metrics per document context), and Long-Term Memory (validated policy knowledge stored as Beta distribution parameters with vector embeddings for semantic retrieval). The architecture must support autonomous promotion of reliable strategies from MTM to LTM, and principled decay of stale entries, without manual intervention.

Objective 2: Develop the RABI algorithm for cold-start transfer.

Develop the Retrieval-Augmented Bandit Initialisation (RABI) algorithm to solve the cold-start problem when new document types or vendors are encountered. RABI must retrieve the k-nearest neighbours from LTM using cosine similarity over 384-dimensional sentence embeddings (generated by all-MiniLM-L6-v2), compute softmax-weighted prior pseudocounts with a configurable temperature parameter tau that controls the sharpness of the weighting, and initialise new arms with an informed Beta prior rather than the uninformative Beta(1,1). The algorithm must include safeguards: a similarity threshold to filter irrelevant neighbours, a minimum sample requirement to avoid transferring from undertrained contexts, and a cap on total transfer strength to prevent overconfident initialisation.

Objective 3: Develop the DAAF mechanism for adaptive forgetting.

Develop the Drift-Aware Adaptive Forgetting (DAAF) mechanism to address concept drift in non-stationary environments. DAAF must maintain dual sliding windows of recent reward observations, compute the drift magnitude as the mean shift between windows - explicitly separating this from within-window noise variance - classify drift as none, gradual, or abrupt based on configurable thresholds, and derive

an optimal forgetting rate gamma from the bias-variance minimisation formula: gamma = sqrt(delta² / ln(t)). The forgetting must be applied to the Beta distribution pseudocounts in a way that preserves the interpretation of pseudocounts as accumulated evidence. For abrupt drift, an additional halving of pseudocounts must accelerate re-exploration.

Objective 4: Integrate AHMN with Thompson Sampling.

Integrate AHMN with a Thompson Sampling policy selector that uses Beta-Bernoulli posteriors, supports both deterministic (expected value) and stochastic (sampling-based) selection modes, and incorporates RABI-initialised priors blended with per-neighbour similarity scores. The confidence metric for each recommendation must combine RABI prior confidence (40%), contextual similarity (30%), and evidence strength (30%), capped at 0.95 to prevent overconfidence.

Objective 5: Implement autonomous memory lifecycle management.

Implement a background scheduler that autonomously manages memory lifecycle: promoting well-performing strategies from MTM to LTM every 30 minutes (threshold: F1 > 0.50, minimum 2 observations), and running DAAF-based forgetting every 6 hours - decaying confidence and pseudocounts of inactive policies, and archiving and removing policies below a confidence threshold of 0.35.

Objective 6: Conduct comprehensive evaluation and ablation studies.

Evaluate the AHMN against baseline algorithms - Random, Epsilon-Greedy (with exponential decay), UCB1, and standard Thompson Sampling - as well as ablated variants (RABI-only, DAAF-only) across four experimental scenarios: stationary, cold-start, concept drift, and combined. Ablation studies must characterise the sensitivity of RABI to temperature (tau in {0.1, 0.3, 0.5, 0.7, 1.0, 2.0}) and similarity threshold, and the sensitivity of DAAF to window size (in {10, 20, 30, 50, 75, 100}). A five-sweep robustness study must identify the boundary conditions under which the AHMN provides measurable benefit over standard Thompson Sampling.

Objective 7: Situate AHMN within the broader multi-agent system.

Situate the AHMN within the broader modular multi-agent system, where the 13-agent IDPO pipeline handles document ingestion through to structured output, the six-layer Guardrails ensemble validates outputs, and the Explainable AI (XAI) module provides human-in-the-loop feedback that feeds back into AHMN's LTM via graduated Beta penalties. The AHMN serves as the system's adaptive memory backbone, receiving policy requests from IDPO, returning contextualised recommendations, and learning from processing outcomes reported by downstream agents.

2. METHODOLOGY

2.1 System Architecture

The system is deployed as eight containerised services orchestrated through Docker Compose, communicating over an internal bridge network with no direct internet access for backend services. A separate external network exposes only the API Gateway (port 8000) and the Next.js frontend (port 3000) to the host. This section briefly describes each microservice to establish how the AHMN core - the primary contribution of this work - integrates with the surrounding system.

AHMN Core (port 8001).

The Adaptive Hierarchical Memory Network is implemented as a standalone FastAPI application responsible for all policy recommendation, outcome logging, drift detection, and memory lifecycle management. It connects to PostgreSQL via psycopg with an asynchronous connection pool (min_size=2, max_size=20) for persistent memory tiers and to Redis via aioredis for volatile short-term storage. On startup, the service registers the pgvector extension for vector similarity queries, seeds Long-Term Memory with 21 initial policy entries if empty, and launches a background scheduler for autonomous promotion and forgetting cycles. The service exposes four endpoint groups: (a) Advice - POST /advice for policy recommendation, POST /advice/log for outcome logging, POST /advice/feedback for human feedback, and POST /advice/review for review feedback from AI or human inspection; (b) Scheduler - GET /scheduler/status for background job monitoring and POST /scheduler/promote for manual promotion trigger; (c) Dashboard - GET /dashboard/stats, GET /dashboard/recent-outcomes, GET /dashboard/ltm-policies, and GET /dashboard/drift-status for memory inspection and analytics; and (d) Experiments - GET /experiments/ for pre-computed evaluation results and interactive demonstrations.

IDPO Pipeline (port 8004).

The Intelligent Document Processing Orchestration pipeline comprises 13 specialised agents: Intake, Pre-processing, Document Analysis, Categorisation, Degarbling, Chunking, Retrieval, LLM Extraction, Structuring, Validation, Policy Enforcement, Escalation, and Recovery. These agents are coordinated by a central

Pipeline orchestrator that executes a dynamically constructed execution plan. The IDPO service communicates with AHMN via synchronous HTTP calls: at the start of processing, it sends a POST /advice request containing the document type, vendor identifier, and content profile, receiving a policy recommendation with a confidence score. After processing completes, it reports the outcome via POST /advice/log with the F1 score, cost, and processing time, closing the feedback loop that enables AHMN to learn.

Guardrails Service (port 8002).

This service runs a six-layer validation ensemble against extracted document data. The checks - schema adherence via Pydantic model validation, relevance through hallucination detection using semantic similarity, internal consistency via business logic rules, domain rules covering tax plausibility and fraud patterns, PII leakage detection using Presidio, and faithfulness computed as weighted semantic plus numerical similarity - execute sequentially with error isolation. Each check returns a score and a pass/fail verdict. The ensemble's results are logged to the guardrail_audit_log table and reported back to IDPO.

XAI-HITL Service (port 8003).

The Explainable AI and Human-in-the-Loop service uses a BART encoder-decoder model (vblagoje/bart_lfqa) for generative question answering over financial documents, with cross-attention extraction and LIME-based perturbation analysis for explainability. Human reviewers can submit feedback - correct, incorrect, partially correct, or needs more context - through the frontend. This feedback is stored in the xai_hitl_feedback table and forwarded to the AHMN's /advice/feedback endpoint, where it adjusts LTM Beta parameters via graduated penalties: an "incorrect" verdict adds 2.0 to beta, "partially correct" adds 0.5, and "correct" adds nothing.

Supporting Infrastructure.

The PostgreSQL database hosts all persistent tables across the four services. The database schema, initialised via scripts/init.sql, defines tables for authentication (users), AHMN memory tiers (mtm_outcomes, ltm_validated_policies), experiment tracking (experiment_runs), guardrail auditing (guardrail_audit_log), and XAI

feedback (xai_hitl_feedback). Redis serves as the STM store for AHMN and as a session cache for the API Gateway. The Gateway handles authentication, CORS, rate limiting, and request routing to downstream services, forwarding user identity headers that AHMN uses for audit trails.

2.2 Adaptive Hierarchical Memory Network (AHMN)

The AHMN implements a three-tier memory architecture inspired by Complementary Learning Systems (CLS) theory [14], where new experiences are first captured rapidly in volatile storage and progressively consolidated into durable long-term knowledge as evidence accumulates. Each tier serves a distinct function in the memory lifecycle, and data flows unidirectionally from STM through MTM to LTM, with the exception of human feedback which can directly modify LTM entries.

2.2.1 Short-Term Memory (STM) - Redis

STM is implemented as a Redis key-value store with a time-to-live (TTL) of 1800 seconds (30 minutes). When the IDPO pipeline requests a policy recommendation via POST /advice, the AHMN generates a UUID called advice_id, stores the full document context and the hash of the selected LTM entry in Redis under the key advice:{advice_id}, and returns the recommendation along with this identifier. The TTL ensures that the context remains available long enough for the IDPO pipeline to complete processing and report outcomes, but is automatically purged to prevent unbounded memory growth. Redis was chosen for STM because of its sub-millisecond read/write latency and native TTL support, which makes it well-suited for transient, high-throughput correlation data.

The STM entry contains the original document context (doc_type, vendor, content_profile, and any extra context) and the source_hash linking back to the LTM entry that produced the recommendation. This linkage is critical: when the outcome is later reported via POST /advice/log, the system retrieves the STM entry to determine which LTM policy should receive the Bayesian update. If the STM entry has expired before the outcome arrives, the log endpoint returns a 404, and the observation is lost - a deliberate design choice that bounds the system's exposure to stale correlations.

2.2.2 Mid-Term Memory (MTM) - PostgreSQL mtm_outcomes

MTM stores every processing outcome as a persistent record in the mtm_outcomes table. The schema is presented in Table 2.1 below.

Table 2.1: Mid-Term Memory (MTM) Schema - mtm_outcomes table

Column	Type	Purpose
id	UUID (PK)	Unique record identifier
advice_id	UUID	Links to the original STM advice entry
policy_id_used	TEXT	The processing policy that was actually applied
document_context	JSONB	Full document metadata (doc_type, vendor, content_profile)
outcome_metrics	JSONB	Processing results (f1_score, cost, processing_time_ms, precision, recall)
human_corrected	BOOLEAN	Whether a human reviewer overrode the result
created_at	TIMESTAMPTZ	Timestamp for temporal ordering and analysis

MTM serves two functions. First, it is the raw evidence store from which the promotion scheduler aggregates statistics to decide whether a context-policy mapping deserves elevation to LTM. Second, it provides the historical reward stream from which the DAAF mechanism estimates per-context drift rates during forgetting cycles. The table is indexed on created_at DESC and policy_id_used to support both recency-ordered dashboard queries and the grouped aggregation queries used by the promotion job.

The human_corrected flag is set by the /advice/feedback endpoint when a reviewer flags an incorrect result through the XAI-HITL interface. This flag is not used directly by the promotion logic - instead, the feedback endpoint applies a graduated Beta penalty to the corresponding LTM entry immediately, providing a faster correction signal than waiting for the next promotion cycle.

2.2.3 Long-Term Memory (LTM) - PostgreSQL with pgvector

LTM is the system's persistent knowledge store, implemented as the ltm_validated_policies table. The schema is presented in Table 2.2 below.

Table 2.2: Long-Term Memory (LTM) Schema - ltm_validated_policies table

Column	Type	Purpose
id	UUID (PK)	Unique record identifier
context_hash	TEXT (UNIQUE)	SHA-256 hash of (context_string + policy_id)
context_embedding	vector(384)	Sentence embedding for cosine similarity search
policy_id	TEXT	The processing policy this entry represents
policy_data	JSONB	Policy metadata (doc_type, vendor, content_profile, action, origin)
validation_metrics	JSONB	Performance history (f1_score, sample_size, source)
alpha	FLOAT	Beta distribution success parameter (accumulated successes)
beta	FLOAT	Beta distribution failure parameter (accumulated failures)
confidence_score	FLOAT	Overall confidence estimate for this entry [0, 1]
last_promoted_at	TIMESTAMPTZ	Last time this entry was updated or promoted from MTM

Each LTM entry represents a learned mapping from a document context to a processing policy, along with the accumulated evidence for that mapping expressed as Beta distribution parameters. The context_embedding column stores a 384-dimensional vector generated by the all-MiniLM-L6-v2 SentenceTransformer model, enabling nearest-neighbour retrieval via pgvector's cosine distance operator. The context_hash column serves as a unique key for each (context, policy) pair, computed as the SHA-256 hash of the concatenation of the context string and the policy identifier.

The embedding is generated by the EmbeddingService, which is implemented as a thread-safe singleton to ensure the SentenceTransformer model is loaded only once. The create_context_string method constructs a descriptive string from the document type and vendor - for example, "Document type: invoice. Processing invoice documents. Vendor: Acme Corp." - which provides rich semantic content for the

embedding model to differentiate between contexts. If a content_profile such as "table-heavy", "form-centric", or "scanned" is available, it is appended to the context string before embedding to further improve semantic separation.

To address the cold-start problem at system level, the AHMN seeds LTM with 21 initial policy entries on first startup if the table is empty (configurable via the AHMN_SEED_MODE environment variable). These seeds cover 11 document types mapped to 11 processing policies: table_heavy (invoice, receipt, table_document, financial_statement), table_dense_local (invoice, receipt, table_document), forms (form_document, clinical_summary, medical_record), forms_rescue_local (form_document, clinical_summary, medical_record), scanned_text (scanned_document), mixed_content (mixed_document, contract), noisy_ocr_rescue (invoice with noise), noisy_ocr_precision_local (invoice with noise), long_document_chunked (contract, long), fast_local_text (text_document), and unknown_vendor_safe (unknown). Each seed is initialised with informative Beta priors reflecting domain knowledge. For example, the invoice-to-table_heavy mapping uses Alpha=9.0, Beta=2.0, encoding a prior belief that this policy succeeds approximately 82% of the time, computed as 9/(9+2) = 0.818. This seed data ensures the system can provide informed recommendations from its very first request.

When the /advice endpoint receives a request, it generates an embedding for the incoming document context and executes a pgvector nearest-neighbour query ordered by cosine distance, retrieving the top-k entries (k=5 by default). The query returns the policy identifier, policy data, Alpha and Beta parameters, context hash, and the cosine similarity score computed as 1 minus the cosine distance. These results are passed to the RABI initialiser and Thompson Sampling selector.

The promotion mechanism operates through two pathways. The first is inline promotion, which occurs within the /advice/log endpoint: when a processing outcome is logged with a reward (F1 score) of 0.40 or higher and no existing LTM entry matches the context-policy pair, a new LTM entry is created immediately with initial priors scaled by the observed reward quality. The second pathway is batch promotion, run by the AHMNScheduler every 30 minutes: it queries MTM for context-policy combinations with an average F1 above 0.50 and at least 2 observations, generates an

embedding for each qualifying combination, and inserts or updates the corresponding LTM entry with accumulated Alpha and Beta values proportional to the average F1 and sample count.

2.3 Algorithmic Implementation

This section details the three core algorithms that govern policy selection, cold-start transfer, and adaptive forgetting within the AHMN. Each algorithm is implemented as a standalone service class in the ahmn_core/services/ package, maintaining clear separation of concerns.

2.3.1 Thompson Sampling for Policy Selection

Thompson Sampling [22] provides the core mechanism by which the AHMN selects processing policies. The approach models each policy's success probability as a Beta distribution and selects policies by sampling from these distributions, naturally balancing exploration of uncertain policies with exploitation of known good ones.

For each context-policy pair stored in LTM, the system maintains two parameters: Alpha (accumulated successes plus prior) and Beta (accumulated failures plus prior). Together, these define a Beta(Alpha, Beta) posterior distribution over the true success probability theta of that policy in that context. The expected value is E[theta] = Alpha / (Alpha + Beta), and the variance is Var[theta] = (Alpha × Beta) / ((Alpha + Beta)² × (Alpha + Beta + 1)). The uninformative starting prior is Beta(1, 1), equivalent to Uniform(0, 1), encoding maximum uncertainty.

The /advice endpoint supports two selection modes, configurable via the AHMN_ADVICE_SELECTION_MODE environment variable. In thompson mode (the default), the system draws a sample from Beta(blended_alpha, blended_beta) for each candidate using Python's random.betavariate, and the candidate with the highest draw wins. Reproducibility is maintained by seeding the random state deterministically from the input context: ctx_seed = int(SHA256(context_str), 16) mod 2^31. In deterministic mode, the system instead computes the expected value E[theta] = blended_alpha / (blended_alpha + blended_beta) for each candidate and selects the highest, ensuring identical inputs always produce identical outputs.

The raw Alpha and Beta from each retrieved LTM entry are not used directly. Instead, they are blended with the RABI-initialised prior, weighted by the cosine similarity between the query context and each neighbour. The blending formula is: blended_alpha = max(0.1, prior.alpha + (alpha_i - 1) × sim_i) and blended_beta = max(0.1, prior.beta + (beta_i - 1) × sim_i). This formulation transfers the pseudocount excess - the evidence accumulated beyond the flat prior - scaled by the contextual similarity. A highly similar neighbour contributes nearly its full evidence, while a distant neighbour contributes proportionally less. The floor of 0.1 prevents degenerate distributions.

The confidence returned with each recommendation depends on whether RABI transfer was applied. On the cold-start path (RABI was used), the formula is: confidence = min(0.95, prior.confidence × 0.4 + similarity_score × 0.3 + evidence_strength × 0.3), where evidence_strength = min(1.0, transferred / 10.0) and transferred is the total pseudocount excess transferred by RABI. On the warm-start path (a direct LTM match existed), the formula is: confidence = min(0.95, 0.4 + similarity_score × 0.3 + evidence_strength × 0.3), where evidence_strength = min(1.0, (alpha + beta - 2.0) / 20.0). In both paths the weights are: RABI prior confidence 40%, contextual similarity 30%, evidence strength 30%. The cap at 0.95 prevents the system from expressing absolute certainty, preserving a margin for human override.

When a processing outcome is reported via /advice/log, the Bayesian update uses a blended reward signal rather than the raw F1 score. Starting from f1_score as the base, the _blended_reward() function adds 0.05 if the output passed schema validation, subtracts up to 0.15 (at 0.05 per field) for missing required fields, subtracts up to 0.25 (at 0.06 per field) for hallucinated fields, adds up to 0.15 for grounding coverage, subtracts 0.10 if defer_status is "requested" and 0.15 if it is "blocked", and subtracts up to 0.08 for processing latency above 2,500 ms. The result is clipped to [0.0, 1.0]. The LTM entry is then updated as alpha_new = alpha_old + reward and beta_new = beta_old + (1 - reward), a standard Beta-Bernoulli update generalised to continuous rewards in [0, 1].

Three baseline algorithms are implemented for experimental comparison. UCB1 [2] selects the arm maximising UCB(i) = mean_reward(i) + sqrt(c × ln(t) / n(i)),

where c=2.0 is the theoretical optimum derived from Hoeffding's inequality, t is the total round count, and n(i) is the pull count for arm i. Epsilon-Greedy [21] explores randomly with probability epsilon(t) = max(0.01, 1.0 × exp(−0.003 × t)), decaying from pure exploration to a 1% floor. Random selection serves as a lower-bound baseline with expected linear regret.

2.3.2 RABI Cold-Start Transfer

Retrieval-Augmented Bandit Initialisation (RABI) addresses a specific limitation of standard Thompson Sampling: when a new document type or vendor is encountered for the first time, the system has no prior evidence and must initialise the arm with an uninformative Beta(1,1) prior. This forces the system to explore all policies from scratch, incurring preventable regret if similar contexts with known performance characteristics already exist in LTM. RABI replaces this flat prior with an informed one, constructed from the nearest neighbours in the embedding space.

Let $N = \{n_1, n_2, ..., n_m\}$ be the set of valid neighbours retrieved from LTM, where each neighbour n_i has Beta parameters ($alpha_i$, $beta_i$), cosine similarity sim_i to the query context, and $n\_samples_i$ observed outcomes. The RABI prior is computed as: $alpha\_init = 1 + \Sigma_i(w_i \times (alpha_i - 1))$ and $beta\_init = 1 + \Sigma_i(w_i \times (beta_i - 1))$. The critical design decision is to transfer the pseudocount excess ($alpha_i - 1$) rather than the raw $alpha_i$. This preserves the interpretation of pseudocounts: in a Beta distribution, the parameters alpha and beta can be understood as (1 + number of observed successes) and (1 + number of observed failures), respectively. Subtracting 1 extracts the evidence component, which is then weighted and re-added to the base prior of 1. If the neighbours collectively provide no evidence, the formula correctly reduces to the flat Beta(1, 1) prior.

The weights w_i are computed via a temperature-scaled softmax over the cosine similarities: $w_i = \exp(sim_i / tau) / \Sigma_j(\exp(sim_j / tau))$, where tau is the temperature parameter. The temperature controls the sharpness of the weight distribution: at low tau (e.g., 0.1), the weight is concentrated almost entirely on the most similar neighbour, while at high tau (e.g., 2.0), the weights are distributed more uniformly across all neighbours. For numerical stability, the implementation subtracts the maximum scaled

similarity before exponentiation - the log-sum-exp trick. The default temperature of tau = 0.5 was selected based on ablation experiments over tau in {0.1, 0.3, 0.5, 0.7, 1.0, 2.0}, balancing the informativeness of the transferred prior against the risk of over-reliance on a single neighbour.

The implementation includes four safeguards against pathological transfer. First, a similarity threshold (default: 0.4) excludes neighbours with cosine similarity below this value, preventing transfer from semantically dissimilar contexts. Second, a minimum sample requirement (default: 5 observations) excludes neighbours observed fewer than 5 times, as their Alpha and Beta values are dominated by the prior rather than genuine evidence. Third, a maximum transfer strength cap (default: 10.0 pseudocounts) limits the total absolute transfer - if the raw transfer exceeds this limit, it is linearly scaled down, preventing a single highly-experienced neighbour from suppressing exploration. Fourth, graceful degradation ensures that if no neighbours pass all three filters, RABI returns a flat Beta(1, 1) prior with confidence 0.0, and the system falls back to standard uninformative Thompson Sampling.

RABI computes a confidence score for the transferred prior as: confidence = min(1.0, avg_similarity × ln(1 + total_samples) / 5). This metric increases with both the average similarity of the neighbours and the logarithm of their total sample count. The logarithmic scaling prevents large sample counts from dominating the confidence - the difference between 100 and 10,000 samples is less important than the difference between 5 and 50.

Under a Lipschitz continuity assumption on the reward function - that |mu(x) - mu(x')| is bounded by L × ||x - x'|| for all contexts x and x' - the expected error of the transferred prior is bounded by L × d_avg, where d_avg is the average embedding distance to the neighbours. When d_avg is small the transferred prior is near-accurate and cold-start regret is reduced from O(K) to O(L × d_avg). When d_avg is large the safeguards prevent harmful transfer and the system falls back to standard Thompson Sampling. In production the RABI-transferred component decays as the policy accumulates direct observations of its own, so the system does not remain dependent on borrowed priors once sufficient evidence is available. The decay rate is set by RABI_TRANSFER_DECAY_RATE (default: 0.05) and the retention factor is

retention(t) = (1 - 0.05)^observed_count, giving effective_alpha = base_alpha × retention and effective_beta = base_beta × retention. The rabi_transfer state is persisted in the validation_metrics JSONB field and updated on every /advice/log call. On each new observation the service separates own evidence as own_successes = alpha - 1 - current_transfer_alpha, advances the transfer count, and reconstructs alpha = 1 + own_successes + next_transfer_alpha. This decomposition preserves the policy's own accumulated evidence correctly as the transferred component fades.

2.3.3 Drift-Aware Adaptive Forgetting (DAAF)

Standard Thompson Sampling accumulates evidence indefinitely, which is optimal in stationary environments but becomes a liability when the underlying reward distributions change. Once an arm has accumulated hundreds of observations, the posterior becomes tightly concentrated, and even a genuine shift in the true reward rate requires many contrary observations to move the posterior significantly. DAAF addresses this by applying a controlled forgetting operation before each Bayesian update, with the forgetting rate dynamically calibrated to the magnitude of detected drift.

The fundamental design principle of DAAF is the explicit separation of two sources of reward variability. The first is noise (sigma-squared): the within-window variance of individual reward observations. This is inherent stochasticity - even a perfectly consistent processing policy produces varying F1 scores across documents due to document complexity, OCR quality, and other factors. Noise is not drift and should not trigger forgetting. The second is drift (Delta): the between-window mean shift. This represents a genuine change in the underlying reward distribution - for example, a vendor updating their invoice template, causing a previously effective OCR engine to degrade. Drift should trigger proportional forgetting.

The DriftEstimator class maintains two sliding windows - a recent_window and an older_window, each of configurable size (default: 50 observations). As new rewards arrive, the oldest element of the recent window is shifted into the older window. The drift magnitude is computed as: Delta = |mean(recent_window) - mean(older_window)|. The noise variance is computed within the recent window alone

using running sum and sum-of-squares - an online variance computation that avoids storing all individual values.

Before classification, the implementation applies a Welch z-test to assess statistical significance. The standard error of the mean difference is SE = sqrt(var_recent / n_recent + var_older / n_older), and the z-score is z = raw_delta / max(SE, 1e-9). Classification requires both statistical significance and a minimum raw magnitude. A z-score below 2.0 or a raw_delta below 0.05 produces a None classification, holding the forgetting rate at its floor of 0.001. A z-score of at least 2.0 combined with raw_delta of at least 0.05 yields a Gradual classification, with gamma* computed from the bias-variance formula. A z-score of at least 3.0 combined with raw_delta of at least 0.15 yields an Abrupt classification, with gamma* doubled and capped at 0.5. The thresholds correspond approximately to $p < 0.05$ and $p < 0.003$ respectively, reducing false positives on stable reward streams. The DriftMetrics dataclass retains both drift_magnitude (post-significance, used for gamma*) and raw_delta alongside the z_score. These magnitude cutoffs align with the scale of F1 variation: a shift below 0.05 falls within typical within-window noise, while a shift of 0.15 or more reflects a meaningful change in processing effectiveness.

The optimal forgetting rate is derived from a bias-variance trade-off. In non-stationary environments, a small gamma retains more historical evidence (lower variance) but is slow to adapt to changes (higher bias from stale data). A large gamma discards historical evidence aggressively (lower bias) but operates with fewer effective observations (higher variance). The total error can be expressed as: Total Error = $\text{Bias}^2 + \text{Variance} = (\text{Delta} / \text{gamma})^2 + \text{sigma}^2 / (t \times \text{gamma})$. Taking the derivative with respect to gamma and setting it to zero yields the simplified form: gamma* = $\text{sqrt}(\text{Delta}^2 / \ln(t))$, where Delta is the estimated drift magnitude and t is the total number of observations. This formula has the desirable properties that gamma* increases with drift magnitude, decreases with time, and is zero when Delta is zero. The implementation bounds gamma* to [0.001, 0.3] to prevent both complete stagnation and overly aggressive memory erasure. For abrupt drift, the bounded value is further doubled (up to a cap of 0.5) to accelerate re-exploration.

The forgetting rate gamma is applied to the Beta distribution parameters as follows: alpha_new = 1 + (1 - gamma) × (alpha_old - 1) and beta_new = 1 + (1 - gamma) × (beta_old - 1). This formula operates on the pseudocount excess, preserving the base prior of 1. The retention factor (1 - gamma) multiplies the accumulated evidence. After the forgetting step, the standard Bayesian update (alpha += reward, beta += 1 - reward) is applied, so fresh observations are always fully incorporated. For abrupt drift specifically, an additional halving operation cuts the effective sample size by half, forcing the system into a near-exploratory state where Thompson Sampling's natural exploration incentive can rapidly discover the new optimal policy.

In the production service, DAAF operates at two levels. Inline (per-request): within the /advice/log endpoint, a DriftEstimator is maintained for each document context key (doc_type:vendor). When an outcome is logged, the estimator is updated with the observed reward, and the computed gamma is applied to the matching LTM entry's Alpha and Beta before the Bayesian update. This ensures that drift is tracked and responded to in real time. Batch (scheduled): the AHMNScheduler runs a forgetting cycle every 6 hours, querying MTM for per-context reward streams from the last 7 days, feeding them through a temporary DriftEstimator for each context, and using the resulting gamma values to decay the confidence scores and Alpha/Beta parameters of inactive LTM entries. The batch forgetting formula is asymmetric and operates on raw Alpha/Beta rather than pseudocount excess: decay_alpha = max(1.0, alpha × (1.0 - decay_rate × 0.5)) and decay_beta = max(1.0, beta × (1.0 - decay_rate × 0.3)), so alpha decays more rapidly than beta. Confidence is updated as new_conf = confidence × (1.0 - decay_rate). Entries whose confidence falls below 0.35 are archived to MTM and removed from LTM. Where a context has fewer than five recent rewards, the base decay rate of 0.05 is applied; otherwise the DriftEstimator's optimal_gamma is used.

2.3.4 Human Feedback (via /advice/feedback)

Human reviewers submit structured feedback through the XAI-HITL frontend via POST /advice/feedback. The endpoint applies graduated Beta penalties to the corresponding LTM entry based on the feedback verdict. An "incorrect" verdict contributes alpha_delta=0.0, beta_delta=2.0, penalising the policy strongly. A

"partially_correct" verdict contributes 0.5 to both alpha and beta, adding weak evidence to each side. A "correct" verdict contributes alpha_delta=1.0, beta_delta=0.0, reinforcing the policy. A "needs_more_context" verdict leaves the parameters unchanged. To illustrate the practical effect: a single "incorrect" verdict on an entry with Alpha=10, Beta=3 shifts the posterior mean from 0.77 to 0.67, a correction that would otherwise require approximately four to five poor automated outcomes to achieve through the standard Bayesian update path.

2.3.5 Review Feedback (via /advice/review)

Beyond human reviewers, the /advice/review endpoint accepts structured extraction evaluations from either automated AI review or human inspection downstream of the IDPO pipeline. The ReviewFeedbackRequest schema includes review_source ("ai" or "human"), a review_weight scalar, confusion-matrix counts (tp, fp, fn), precision, recall, f1, field_state_counts, and optional job_id and overlay_key fields. On receipt the endpoint logs the review in MTM, computes alpha_delta = review_weight × f1 and beta_delta = review_weight × (1 - f1), then updates the matching LTM entry or creates a new one through inline promotion if no entry exists. This path complements the /advice/feedback route by accepting fine-grained field-level metrics rather than a coarse verdict label.

2.4 Supporting Subsystems

This section briefly describes the Guardrails and XAI-HITL subsystems, focusing on their interaction with the AHMN rather than their internal design.

The six-layer guardrail ensemble validates every document processing output before it reaches the end user. The checks execute in sequence, each isolated from failures in preceding checks. Schema adherence validates extracted JSON against Pydantic models specific to each document type. Relevance computes semantic similarity between extracted fields and the source document contexts using the all-MiniLM-L6-v2 embedding model - low similarity flags potential hallucinations. Internal consistency applies business logic rules such as arithmetic checks (do line items sum to the total?), temporal checks (are dates in the future?), and domain-specific constraints. Domain rules check for tax plausibility, vendor fraud indicators,

suspicious keywords, and unusually large monetary amounts. PII leakage uses Presidio-based detection with context-aware rules to identify personally identifiable information that should not appear in extraction outputs. Faithfulness computes a weighted combination of semantic similarity, numerical accuracy, and entity overlap - this is the most computationally intensive check, capped at a 30-second timeout.

For the AHMN, the guardrail pass/fail outcome contributes to the F1 score reported in the outcome metrics - a document that fails guardrails receives a lower effective reward, which penalises the responsible policy in LTM. This creates an indirect feedback loop between output quality and policy selection without requiring explicit guardrail awareness in the AHMN's learning logic.

The XAI-HITL module provides two forms of explainability for question-answering operations over financial documents. First, cross-attention maps extracted from the BART encoder-decoder's final decoder layer reveal which encoder (source document) tokens the model attended to when generating each answer token. Second, LIME performs perturbation-based analysis by systematically masking input tokens and observing the effect on the model's output probability, producing feature importance scores independent of the model's internal attention patterns.

Human reviewers interact with these explanations through the frontend and can submit structured feedback along with optional corrected answers and quality ratings. This feedback is forwarded to the AHMN's /advice/feedback endpoint. The AHMN applies a graduated Beta penalty: beta += 2.0 for "incorrect," beta += 0.5 for "partially correct," and no penalty for "correct" or "needs more context." This mechanism creates a closed feedback loop between human judgement and the AHMN's Bayesian knowledge base, enabling the memory network to incorporate qualitative assessments alongside quantitative F1 metrics.

2.5 Commercialisation Aspects

The AHMN's adaptive memory architecture offers tangible benefits across several regulated industries where document processing accuracy is critical and operational conditions are non-stationary.

Healthcare.

Medical document processing involves diverse formats - clinical summaries, discharge notes, lab reports, radiology findings, and insurance forms - each requiring different extraction strategies. The LTM seed data includes mappings for clinical summaries and medical records to the forms processing policy, reflecting the structured, field-based nature of these documents. In practice, healthcare providers frequently change electronic health record (EHR) systems, which alters the format and structure of generated documents. DAAF enables the system to detect these template shifts and adjust processing policies without manual reconfiguration. Additionally, the PII leakage guardrail check is particularly important in healthcare contexts, where patient identifiers must be handled in compliance with regulations such as HIPAA. The AHMN's per-context learning means that the system develops specialised processing knowledge for each healthcare provider's document formats over time, reducing extraction errors that could have clinical consequences.

Finance.

Financial document processing - invoices, receipts, bank statements, financial reports - is characterised by high volume, strict accuracy requirements, and frequent format changes as vendors update their billing systems. The AHMN's LTM contains dedicated seed entries for invoices, receipts, and financial statements, all mapped to the table_heavy processing policy that prioritises tabular data extraction. The RABI algorithm is particularly valuable in this domain: when a new vendor is onboarded, the system immediately transfers knowledge from existing vendors with similar document formats, reducing the initial accuracy degradation that would otherwise occur during the exploration phase. The Thompson Sampling framework provides a principled mechanism for balancing the cost of trying a new, potentially better OCR engine against the safety of using the known good one - a trade-off that has direct financial implications when processing errors lead to payment delays or reconciliation failures.

Procurement.

Procurement workflows involve contracts, purchase orders, supplier invoices, and compliance documents that flow through multiple organisational units. The AHMN's memory hierarchy is well-suited to this domain because procurement relationships are inherently long-lived: the system processes documents from the same set of suppliers repeatedly over months or years, building increasingly accurate processing policies for each supplier's document formats. When a supplier switches to a new invoicing platform - a common occurrence in procurement - the DAAF mechanism detects the resulting accuracy drop, increases the forgetting rate for that supplier's stale policy, and enables rapid convergence on a new optimal strategy. The AHMN's ability to maintain separate context-policy mappings per supplier and document type means that a processing policy that works well for one supplier's invoices will not be inappropriately applied to another supplier's contracts, even if both are classified as "procurement documents" at a higher level.

Across all three domains, the AHMN's background scheduler ensures that the memory network evolves autonomously - promoting successful strategies, decaying stale ones, and archiving forgotten policies - without requiring manual intervention from operations teams. This self-maintaining behaviour reduces the operational overhead of deploying and maintaining document processing systems in production, where the cost of manual policy tuning can exceed the cost of the processing infrastructure itself.

3. RESULTS AND DISCUSSION

3.1 Performance Optimisations

A central objective of the system was to reduce end-to-end document processing latency from a baseline of approximately 320 seconds per document - observed in early iterations where the IDPO pipeline processed documents sequentially, each agent waited synchronously for downstream services, and no cached knowledge was available to guide policy selection - to sub-second response times for the AHMN's policy recommendation component, and single-digit seconds for full pipeline execution.

Three architectural decisions within the AHMN contributed directly to this latency reduction. First, memory-tier routing eliminates redundant computation. Without the AHMN, the IDPO pipeline had to evaluate every available processing policy through trial execution or heuristic scoring before selecting one. With the AHMN in place, the pipeline issues a single HTTP POST to /advice, which executes a pgvector nearest-neighbour query over 384-dimensional embeddings (a sub-millisecond operation on the indexed ltm_validated_policies table), computes softmax-weighted Thompson Sampling scores for at most k=5 candidates, and returns a policy recommendation. The AHMN endpoint is configured with a 500-millisecond timeout in the IDPO integration layer, ensuring that the policy selection step never becomes a bottleneck. This replaced what was previously a multi-second evaluation loop with a single network call that completes in tens of milliseconds under normal conditions.

Second, singleton embedding model avoids repeated model loading. The EmbeddingService is implemented as a thread-safe singleton that loads the all-MiniLM-L6-v2 model exactly once into memory. The model produces embeddings in approximately 50 milliseconds per inference on CPU. Without the singleton pattern, each advice request would incur a model loading penalty of several seconds - a cost that would negate the benefit of memory-based routing entirely.

Third, Redis STM provides sub-millisecond correlation. The advice-outcome correlation - linking the policy recommendation at request time to the processing result

reported minutes later - is stored in Redis with a 1800-second TTL. Redis's in-memory architecture ensures that both the write at advice time and the read at outcome logging time complete in under 1 millisecond. The alternative - storing this correlation in PostgreSQL - would have added disk I/O latency and required periodic cleanup queries, both of which are avoided by Redis's native TTL eviction.

The IDPO pipeline's process_document method records latency_ms for each document, covering the full span from intake through to finalisation. The rollback monitor enforces a p95 latency threshold of 2,500 milliseconds: if a processing policy consistently exceeds this threshold, it is automatically blocked, and the system falls back to the default policy. The reduction from 320 seconds to under 1 second for the policy selection component - and to low single-digit seconds for the full pipeline - represents a reduction of over two orders of magnitude, made possible by replacing brute-force policy evaluation with memory-augmented retrieval.

3.2 Evaluation Metrics

The AHMN was evaluated through a structured experimental framework that compares seven methods across four scenarios, each run with 30 random seeds for statistical reliability. The seven methods are: Random, Epsilon-Greedy, UCB1, standard Thompson Sampling (TS), RABI-only Thompson Sampling, DAAF-only Thompson Sampling, and the full AHMN (RABI + DAAF). The four scenarios are described below.

Stationary (no drift, no cold-start): A fixed set of vendors is established from the start, and reward distributions remain constant throughout the 500 rounds. This scenario tests whether RABI and DAAF impose any cost in a stable environment. The expectation is that the full AHMN should perform comparably to standard Thompson Sampling, since neither RABI transfer nor DAAF forgetting should be activated significantly.

Cold-Start: Six new vendors arrive in waves throughout the experiment, each with no prior entries in LTM. RABI-equipped methods should demonstrate faster convergence to good policies for each new vendor, because they initialise with informed priors transferred from existing LTM entries with similar document types. Standard Thompson Sampling and non-RABI methods must explore from scratch.

Concept Drift: A single vendor undergoes gradual drift (mean reward shifts from 0.85 to 0.60 over 200 rounds) followed by abrupt drift (a sudden jump to a new optimal policy at round 350). DAAF-equipped methods should detect both drift events and increase their forgetting rate accordingly, while standard Thompson Sampling accumulates stale evidence and is slow to adapt.

Combined: Both cold-start and concept drift occur simultaneously, representing the most challenging real-world scenario. The full AHMN (RABI + DAAF) is expected to outperform all ablated and baseline methods, since both mechanisms are needed.

Five primary metrics are used for evaluation. Cumulative Regret is the running sum of (oracle_reward - actual_reward) over all rounds, measuring the total cost of suboptimal decisions. The oracle reward is the true mean of the best policy for the

active vendor at each round. Lower cumulative regret indicates faster convergence. Final Average Reward (F1) is the mean reward over the last 100 rounds, measuring steady-state performance after the algorithm has had sufficient time to learn. Policy Selection Accuracy is the fraction of rounds in which the algorithm selected the oracle-best policy - stricter than reward-based metrics because it requires identifying the single best policy, not merely a good one. Convergence Round is the first round at which the rolling-50 average reward exceeds 95% of the oracle average. Noise-Drift Separation assesses whether DAAF correctly distinguishes stable reward variability from genuine distributional shift.

The robustness boundaries identified by the five-sweep study characterise when the AHMN provides measurable benefit. RABI's improvement increases with cosine similarity between established and new vendors: at low similarity (cosine below approximately 0.5), the transferred prior becomes unreliable and may slightly increase regret relative to standard Thompson Sampling. DAAF provides increasing benefit as drift magnitude grows beyond the detection threshold of 0.05: at zero drift, DAAF imposes a small cost due to unnecessary forgetting. RABI's relevance scales with the proportion of new vendors: at 0% new vendors it is irrelevant; at 50% or higher, its contribution to regret reduction is substantial. DAAF's effectiveness depends on per-vendor sample density: with 2 vendors (approximately 500 samples each), drift detection is reliable; with 20 vendors (approximately 50 samples each), the signal may be too diluted for reliable detection. When new vendors have fundamentally different optimal policies from their embedding-space neighbours, RABI transfers incorrect priors, and the robustness study identifies the crossover point at which RABI begins to hurt rather than help.

3.3 Discussion

The AHMN's closed-loop learning mechanism demonstrates a fundamentally different approach to document processing adaptability compared to static pipeline configurations. Rather than relying on manually tuned rules or periodic offline retraining, the system continuously learns from every processed document through a Bayesian feedback cycle.

Each document processed by the system completes a full learning loop. First, the IDPO pipeline requests a policy recommendation from the AHMN, receiving a Thompson Sampling-selected policy with a confidence score. Second, the pipeline processes the document using the recommended policy. Third, the pipeline reports the outcome (F1 score, cost, processing time) back to the AHMN via /advice/log. Fourth, the AHMN updates the corresponding LTM entry's Beta distribution parameters, applies DAAF-based forgetting if drift is detected, and optionally promotes new context-policy mappings to LTM if the outcome is sufficiently good. This cycle operates continuously and autonomously - no human intervention is required for the system to improve over time.

The three-tier memory architecture enables the system to handle document variability at different time scales. Short-term variability - such as a batch of unusually noisy scans from a particular vendor - is absorbed by the noise variance component of the DAAF estimator and does not trigger forgetting. Medium-term changes - such as a vendor gradually transitioning to a new invoice template over several weeks - produce gradual drift that increases the forgetting rate proportionally, allowing the system to adapt without abruptly discarding its accumulated knowledge. Abrupt changes - such as a vendor switching to an entirely new billing platform - trigger the abrupt drift classification and force the system into a near-exploratory state from which it can rapidly discover the new optimal policy.

The RABI mechanism proves particularly valuable in the document processing domain because of the strong semantic structure in the embedding space. Document types within the same category - for example, invoices from different vendors - tend to cluster in the embedding space produced by all-MiniLM-L6-v2, because the context

strings share substantial semantic content. This clustering means that when a new vendor is encountered, RABI reliably retrieves LTM entries from vendors with similar document types, and the transferred priors are likely to be relevant. The 21 seed entries in LTM ensure that even the very first request from a completely new system deployment receives an informed recommendation rather than a random guess.

The graduated Beta penalty mechanism allows human expertise to correct the system's learned knowledge without requiring full retraining. A single "incorrect" feedback on an LTM entry with Alpha=10, Beta=3 shifts the expected value from 0.77 to 0.67 - a meaningful correction that the system would otherwise require approximately four to five poor outcomes to learn on its own. This asymmetry between the cost of human feedback (one interaction) and the cost of automated learning (multiple poor outcomes) makes the HITL pathway an efficient correction mechanism.

The system's reliance on the all-MiniLM-L6-v2 embedding model means that the quality of semantic similarity - and therefore the quality of RABI transfer - is bounded by the model's ability to differentiate document contexts. The context strings used for embedding are relatively simple (document type, vendor name, content profile) and may not capture fine-grained structural differences between document formats that affect processing policy effectiveness. Additionally, the DAAF mechanism's dual-window design requires a minimum of approximately 60 observations before it can reliably detect drift, which means that very low-volume document contexts may not accumulate enough evidence for adaptive forgetting to activate. These are acknowledged limitations that future work should address.

3.4 Summary of My Contribution

My individual contribution to this research project centred on the design and full implementation of the AHMN Core service - the adaptive memory backbone that enables the broader multi-agent system to learn from experience.

I engineered the AHMN as a standalone FastAPI microservice, implementing the complete three-tier memory database architecture from the ground up. This involved designing the PostgreSQL schema for Mid-Term Memory (the mtm_outcomes table) and Long-Term Memory (the ltm_validated_policies table with pgvector embeddings), integrating Redis as the Short-Term Memory store with TTL-based eviction, and building the asynchronous connection management layer using psycopg's AsyncConnectionPool with pgvector type registration. I wrote the database initialisation script (scripts/init.sql) that creates the tables, indexes, and the pgvector extension.

I implemented the Thompson Sampling policy selector, including both the deterministic (expected value) and stochastic (Beta-variate sampling) selection modes, the deterministic seeding mechanism that ensures reproducibility from the same input context, and the similarity-weighted blending of RABI priors with per-neighbour evidence. I designed the composite confidence metric that combines RABI prior confidence, contextual similarity, and evidence strength.

I designed and implemented the RABI algorithm as a complete solution to the cold-start problem. This included the mathematical formulation of the softmax-weighted pseudocount transfer, the four safeguard mechanisms (similarity threshold, minimum sample requirement, transfer strength cap, and graceful degradation), and the confidence metric based on average similarity and log-scaled sample count. I also implemented the RABIThompsonSampling class with optional exponential decay of transferred knowledge.

I designed and implemented the DAAF mechanism, including the DriftEstimator class with its dual sliding-window architecture, the online variance computation for noise-drift separation, the drift classification logic (none, gradual, abrupt), and the bias-variance-derived optimal forgetting rate formula. I implemented

both the inline DAAF integration within the /advice/log endpoint and the batch DAAF integration within the background scheduler.

I built the AHMNScheduler that runs autonomous promotion (MTM to LTM, every 30 minutes) and forgetting (DAAF-based decay and archival, every 6 hours) as background asyncio tasks, including the per-context adaptive decay rate computation from recent MTM reward streams.

I authored the complete experimental evaluation framework: the synthetic environment with four scenarios (stationary, cold-start, concept drift, combined), the seven method runners, the ablation study infrastructure for RABI temperature, DAAF window size, and similarity threshold, and the five-sweep robustness study that honestly characterises the boundary conditions of the AHMN's effectiveness. I also wrote the analysis module with Mann-Whitney U statistical significance tests and LaTeX table generation.

Finally, I wrote the LTM seed data module with 21 domain-informed policy entries covering 11 document types and 11 processing policies, the embedding service singleton, the Pydantic API schemas, and the full test suite covering Thompson Sampling, RABI, and DAAF algorithms.

4. CONCLUSION

This dissertation presented the Adaptive Hierarchical Memory Network (AHMN), a three-tier memory architecture that enables a modular multi-agent document processing system to learn, adapt, and forget processing policies autonomously. The research addressed two specific gaps in the existing literature: the cold-start problem in contextual bandits, where new document types must be explored from scratch, and the challenge of concept drift in non-stationary environments, where previously learned policies become outdated as document formats change.

The RABI algorithm successfully addresses the cold-start problem by transferring Bayesian prior knowledge from semantically similar historical contexts. Through softmax-weighted pseudocount aggregation over nearest-neighbour embeddings, RABI replaces the uninformative Beta(1,1) prior with an informed distribution that reflects the system's accumulated experience with similar document types. The four safeguard mechanisms - similarity threshold, minimum sample requirement, transfer strength cap, and graceful degradation - prevent harmful transfer when the conditions for reliable knowledge transfer are not met. The robustness study confirms that RABI provides measurable regret reduction when neighbour similarity is high and the proportion of new, unseen document types is significant.

The DAAF mechanism addresses concept drift through a principled approach that explicitly separates true distributional shift from inherent reward noise. By computing the optimal forgetting rate from the bias-variance trade-off formula - $\gamma^* = \sqrt{\Delta^2 / \ln(t)}$ - the system adjusts its forgetting intensity proportionally to the magnitude of observed environmental change. The dual classification into gradual and abrupt drift, with corresponding proportional and amplified forgetting responses, enables the system to adapt to both slow template migrations and sudden platform changes. The robustness study confirms that DAAF's benefit increases with drift magnitude and per-context sample density, while imposing only minimal cost in stationary environments.

The three-tier memory architecture - Redis STM for in-flight correlation, PostgreSQL MTM for outcome logging, and pgvector-enabled LTM for validated

policy knowledge - provides a complete memory lifecycle. The background scheduler autonomously promotes successful strategies and decays stale ones, eliminating the need for manual policy management. The integration with a 13-agent IDPO pipeline, a six-layer Guardrails ensemble, and an XAI-HITL feedback loop demonstrates that the AHMN can serve as the adaptive backbone for a complex, multi-service document processing system.

From a performance perspective, the AHMN's memory-based policy routing reduced the policy selection component from a multi-second evaluation process to a sub-second retrieval and scoring operation, contributing to an overall pipeline latency reduction from approximately 320 seconds to single-digit seconds per document. The 500-millisecond timeout constraint and the singleton embedding model ensure that the AHMN never becomes a bottleneck in the broader pipeline.

Future Recommendations

Several directions warrant further investigation following this work. First, the embedding model could be fine-tuned on domain-specific document corpora to improve the semantic differentiation between document contexts, which would directly enhance RABI's transfer accuracy. A model trained on invoice, form, and contract corpora would likely produce tighter intra-class clusters and clearer inter-class separation in the embedding space.

Second, the DAAF mechanism could be extended with a formal change-point detection algorithm, such as CUSUM or Bayesian Online Changepoint Detection [1], to complement the current sliding-window approach with statistically grounded breakpoint identification. The sliding-window design has a lag equal to the window size; a change-point detector could identify abrupt shifts more rapidly.

Third, the system could incorporate multi-objective Thompson Sampling that jointly optimises for F1 score, processing cost, and latency, rather than using F1 as the sole reward signal. In production environments, the trade-off between accuracy and cost is an important operational consideration that a multi-objective bandit formulation would handle naturally.

Fourth, the LTM could be augmented with a graph structure capturing relationships between document types and policies, enabling more structured knowledge transfer beyond embedding-based similarity. A graph-based memory would allow the system to reason about hierarchical relationships - for example, that "pharmaceutical invoices" are a subtype of "invoices" - and exploit these relationships during cold-start transfer.

Finally, a live production deployment study - comparing the AHMN-equipped system against a fixed-policy baseline on real enterprise documents over several months - would provide the definitive evidence of practical value that synthetic experiments, however carefully designed, cannot fully replicate. Such a study would also surface operational concerns not captured in simulation, including distribution shift in document types not represented in the seed data, adversarial inputs, and multi-tenant isolation requirements.

REFERENCES

[1] R. P. Adams and D. J. C. MacKay, "Bayesian online changepoint detection," arXiv preprint arXiv:0710.3742, 2007.

[2] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," Machine Learning, vol. 47, no. 2-3, pp. 235-256, 2002.

[3] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," IEEE Transactions on Neural Networks, vol. 5, no. 2, pp. 157-166, 1994.

[4] C. Blundell, B. Uria, A. Pritzel, Y. Li, A. Ruderman, J. Z. Leibo, J. Rae, D. Hassabis, and G. Wayne, "Model-free episodic control," arXiv preprint arXiv:1606.04460, 2016.

[5] O. Chapelle and L. Li, "An empirical evaluation of Thompson sampling," in Advances in Neural Information Processing Systems, vol. 24, 2011, pp. 2249-2257.

[6] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in Proc. 2014 Conf. on Empirical Methods in Natural Language Processing (EMNLP), 2014, pp. 1724-1734.

[7] R. M. French, "Catastrophic forgetting in connectionist networks," Trends in Cognitive Sciences, vol. 3, no. 4, pp. 128-135, 1999.

[8] A. Garivier and E. Moulines, "On upper-confidence bound policies for switching bandit problems," in Proc. 22nd Int. Conf. on Algorithmic Learning Theory (ALT), 2011, pp. 174-188.

[9] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, "Retrieval augmented language model pre-training," in Proc. 37th Int. Conf. on Machine Learning (ICML), 2020, pp. 3929-3938.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[11] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," Proceedings of the National Academy of Sciences, vol. 114, no. 13, pp. 3521-3526, 2017.

[12] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in Proc. 19th Int. Conf. on World Wide Web (WWW), 2010, pp. 661-670.

[13] F. Liu, J. Lee, and N. Shroff, "A change-detection based framework for piecewise-stationary multi-armed bandit problem," in Proc. AAAI Conf. on Artificial Intelligence, vol. 32, no. 1, 2018, pp. 3651-3658.

[14] J. L. McClelland, B. L. McNaughton, and R. C. O'Reilly, "Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory," Psychological Review, vol. 102, no. 3, pp. 419-457, 1995.

[15] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," in The Psychology of Learning and Motivation, G. H. Bower, Ed., vol. 24. Academic Press, 1989, pp. 109-165.

[16] V. Raj and S. Kalyani, "Taming non-stationary bandits: A Bayesian approach," arXiv preprint arXiv:1707.09727, 2017.

[17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. 2019 Conf. on Empirical Methods in Natural Language Processing (EMNLP), 2019, pp. 3982-3992.

[18] H. Robbins, "Some aspects of the sequential design of experiments," Bulletin of the American Mathematical Society, vol. 58, no. 5, pp. 527-535, 1952.

[19] D. J. Russo, B. Van Roy, A. Kazerouni, I. Osband, and Z. Wen, "A tutorial on Thompson sampling," Foundations and Trends in Machine Learning, vol. 11, no. 1, pp. 1-96, 2018.

[20] A. I. Schein, A. Popescul, L. H. Ungar, and D. M. Pennock, "Methods and metrics for cold-start recommendations," in Proc. 25th Annu. Int. ACM SIGIR Conf. on Research and Development in Information Retrieval, 2002, pp. 253-260.

[21] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. MIT Press, 2018.

[22] W. R. Thompson, "On the likelihood that one unknown probability exceeds another in view of the evidence of two samples," Biometrika, vol. 25, no. 3/4, pp. 285-294, 1933.

[23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems, vol. 30, 2017, pp. 5998-6008.

[24] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "LayoutLM: Pre-training of text and layout for document image understanding," in Proc. 26th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, 2020, pp. 1192-1200.

APPENDIX A: CONFIGURATION REFERENCE

A.1 AHMNConfig vs Actual Runtime Values

Several parameters in ahmn_core/config.py differ from the values actually used at runtime. EPSILON_DECAY_RATE is defined as 0.001 in config but the

experiment runner overrides this with 0.003. PROMOTION_MIN_SAMPLES is set to 10 in config but the scheduler enforces a minimum of 2 observations. PROMOTION_CONFIDENCE_THRESHOLD is 0.70 in config but is not referenced by the scheduler, which applies an F1 threshold of 0.50 instead. FORGETTING_DECAY_DAYS is 30 in config but the scheduler targets policies inactive for 7 days. SIMULATION_NUM_SEEDS is 10 in config but the experiment runner runs 30 seeds for statistical reliability. These discrepancies are noted here so that values stated in the experimental evaluation reflect the code as executed, not as configured.

A.2 Production Environment Variables

Three environment variables control runtime behaviour. AHMN_ADVICE_SELECTION_MODE (default: "thompson") switches between stochastic Thompson Sampling and deterministic expected-value selection. AHMN_SEED_MODE (default: "seeded") controls whether LTM is populated with seed entries on first startup. AHMN_RABI_TRANSFER_DECAY_RATE (default: 0.05) sets the exponential decay rate applied to the RABI transfer component as direct observations accumulate.

A.3 RABI Production Parameters

k (neighbours) = 5; similarity_threshold class default = 0.3, production value = 0.4; temperature (τ) = 0.5; min_samples_for_transfer = 5; max_transfer_strength = 10.0.

A.4 DriftEstimator Parameters

window_size = 50; min_samples = 10; drift_threshold = 0.05; abrupt_threshold = 0.15; γ^* bounds = [0.001, 0.3], doubled for abrupt drift (cap 0.5).

A.5 Scheduler Parameters

Promotion interval: 30 minutes (initial delay 60 s). Forgetting interval: 6 hours (initial delay 90 s). Base decay rate: 0.05. Confidence threshold for removal: 0.35. Inactivity period: 7 days. Drift window for batch rates: 50.